

✦ JOSA //

Building Your Own OSS Project: Licensing & Foundations

/ Choose the right license and set up community standards properly */*

OpenLab Apprenticeship 2026

Week 09

// 1 //

Learning Goals

- Choose an OSS license deliberately, not by accident.
- Understand the difference between **permissive** (MIT, Apache 2.0, BSD) and **copyleft** (GPL, AGPL, MPL).
- Set up the GitHub “community standards” checklist properly: README, LICENSE, CONTRIBUTING, CODE_OF_CONDUCT, SECURITY.
- Pick a versioning scheme and a CHANGELOG strategy.
- Decide between BDFL, do-ocracy, or committee governance and document the decision.

// 2 //

Technical Deep Dive

// 2.1 The license question

Choosing a license is, legally, the single most important decision you make about an OSS project. It determines who can use your code, on what terms, and whether they can keep their modifications private.

Licenses fall into two broad families:

- **Permissive:** Few restrictions. Anyone can use, modify, and redistribute your code, including in closed source or commercial projects, as long as they keep the original copyright notice.
- **Copyleft:** Comes with a condition: if someone distributes a modified version (or, in some cases, runs it as a service), they must release their changes under the same license. This is what keeps derivatives open.

Ask this first: do you care whether someone can take your code, modify it, and never share those changes back?

- **No, I don't care** → pick a permissive license (MIT, Apache 2.0, BSD).
- **Yes, I want derivatives to stay open** → pick a copyleft license (GPL, AGPL, MPL).

License options at a glance

- **MIT:** The simplest. Anyone can do anything with your code, e.g. use it, modify it, sell it, keep changes private as long as they include the original copyright notice and license text somewhere in their distribution (typically a [LICENSE](#) file, or a comment block at the top of the source). Use this for libraries you want widely adopted.
- **Apache 2.0:** Like MIT, but adds an explicit patent grant and patent-retaliation clause (if someone sues you over patents on this code, they lose their license to use it). Use this for non-trivial libraries, especially anything in enterprise or standards-track territory.
- **BSD-3-Clause:** like MIT, but also forbids using the copyright holder's or contributor's names to promote derivative products without permission. Niche; pick MIT or Apache 2.0 instead unless you have a specific reason.
- **GPL-3.0:** Copyleft. If you distribute a combined work that includes GPL code, you must release that combined work under GPL too. Purely private or internal use never triggers this only distribution does. Use this when you want to ensure derivatives stay open. (Linux, GCC.)
- **AGPL-3.0:** like GPL, but the trigger is "use over a network," not just distribution. Closes the SaaS loophole. A company can't host a modified version as a service without releasing their changes. Use this for software you don't want big cloud providers to host without contributing back. (Grafana, Mastodon use AGPL.)
- **MPL-2.0:** File-level copyleft. Modifications to MPL files must be open; new files can be proprietary. A pragmatic middle ground between MIT and GPL. (Firefox uses this.)

A note on dual licensing: some projects offer two licenses at once, commonly a copyleft license (like GPL) paired with a separate commercial license for companies that don't want the copyleft obligations. MySQL and Qt both use this model. It's more complex to administer, so it's rarely the right starting point for a new project, but worth recognizing when you see it.



IMPORTANT

A common confusion: "open source" ≠ "no rules." MIT has rules. They're just very few. **Always include the LICENSE file**, and always include the year and copyright holder name.

The opposite mistake is just as common: **a public repo with no LICENSE file has no rights granted at all.** Visible on GitHub doesn't mean usable — by default, all rights are reserved, and nobody can legally use, copy, or modify your code no matter how public it looks.

The interactive tool choosealicense.com is GitHub's official guide and walks you through the decision in 60 seconds.

// 2.2 Beware “source-available” non-OSS licenses

Some projects use licenses that look like OSS but aren't OSI-approved: **BSL** (Business Source License), **SSPL** (Server Side Public License), **Elastic License**, **Functional Source License**. The OSI (Open Source Initiative) maintains the official definition of "open source" and approves licenses against it and these don't qualify, because they typically carve out one specific use case: **you can't offer the software as a competing hosted/managed service**. Everything else about the code stays visible and modifiable, but that one restriction is enough to disqualify them.

A quick way to spot one in the wild: if a license has language singling out *who can't use it commercially* (usually aimed at cloud providers), it's not OSI-approved, no matter how open the rest of the code looks.

Why projects do this: MongoDB moved from AGPL to SSPL, and Elastic moved from Apache 2.0 to the Elastic License, both specifically to stop cloud providers like AWS from reselling their software as a hosted service without contributing anything back. AGPL didn't fully solve that problem for them, so they wrote licenses that did.

Calling these "open source" is incorrect. The OSI is strict about this, and so should you be. Use them if you have real business reasons, but don't conflate them with true OSS in your communications.

// 2.3 The community standards checklist

GitHub computes a **Community Profile** for every public repo, it's a quick signal to potential contributors and users that a project is actively maintained and safe to invest time in. Hit all of these for a green checkbox:

The quick ones (mostly copy-paste):

- **LICENSE:** The actual file, named exactly LICENSE (not LICENSE.md). Some legal and compliance tooling scans specifically for a plaintext file with that name.
- **CODE_OF_CONDUCT.md:** Use the [Contributor Covenant 2.1](#). Don't write your own.
- **Issue and PR templates:** .github/ISSUE_TEMPLATE/ for bug reports and feature requests, .github/pull_request_template.md for the What/Why/How/Test structure.
- **CODEOWNERS:** .github/CODEOWNERS auto-assigns reviewers based on files touched.

The ones that need real writing:

- **README.md:** What the project is, install, quick example, link to docs, license badge.
- **CONTRIBUTING.md:** The contribution guidelines and how to set up the dev environment, run tests, and submit a PR. This is what new contributors read first; if it's bad, you lose them.

- **SECURITY.md**: How to responsibly report a vulnerability (private email, GitHub Security Advisories, etc.). Don't make people open a public issue for a security bug.

// 2.4 A README that gets people to use your project

The first 8 seconds of someone landing on your README determine whether they'll try the project. Top of the README, in order:

1. **Project name** + one-sentence description.
2. **Badges** (build status, license, version, downloads).
3. **A 5-line code example** showing the simplest "hello world" use.
4. **Install and setup** one line if possible: `npm install foo` / `pip install foo`.
5. **Link to the docs site** (built per Week 6).



TIP

Don't bury the code example. People skim. The example is the conversion event.

// 2.5 Versioning and CHANGELOG

Adopt **SemVer** (Semantic Versioning): version numbers follow MAJOR.MINOR.PATCH, e.g.

1.2.0. Bump **MAJOR** for breaking changes, **MINOR** for new backward-compatible features, and **PATCH** for backward-compatible bug fixes. One exception worth knowing: anything below 1.0.0 is considered unstable, and is allowed to break at any time. A reasonable place to start a brand-new project.

Adopt **Keep a Changelog**. The CHANGELOG.md format:

```
# Changelog

All notable changes to this project will be documented in this file.

## [Unreleased]
### Added
- New `--quiet` flag for the CLI.

## [1.2.0] - 2025-04-01
### Added
- Support for trailing commas in input.
### Fixed
- Crash on empty input (#142).

## [1.1.0] - 2025-02-15
### Changed
```

- Default timeout increased to 30s.

Three sections matter most: **Added**, **Changed**, **Fixed**. Less common but valid: **Removed**, **Deprecated**, **Security**. Each version heading should correspond to an actual git tag (e.g. v1.2.0) and, ideally, a GitHub Release. The CHANGELOG entry and the release notes should say the same thing.

Pair with **release-please** (Google) or **semantic-release** (npm-ecosystem favorite). Both work off **Conventional Commits** (a commit message convention where the prefix tells the tool what changed):

- fix: correct off-by-one in pagination → PATCH bump
- feat: add --quiet flag → MINOR bump
- feat!: remove deprecated config format or a BREAKING CHANGE: footer → MAJOR bump

Follow that convention consistently and the tool reads your commit history, calculates the next version number, writes the CHANGELOG entry, and publishes the release. No manual version bumping.

// 2.6 Governance: BDFL, do-ocracy, committee

When more than one person works on your project, decisions need a process. Three honest models:

- **BDFL (Benevolent Dictator For Life)**: One person has final say. Linus Torvalds for Linux, Guido van Rossum for Python until 2018. Works at small scale; brittle.
- **Do-ocracy**: Whoever does the work makes the call. Common in volunteer projects. Risk: the loudest doer “wins” decisions that should be deliberated.
- **Committee / steering group**: Explicit governance with a documented decision process. The Apache Software Foundation, the CNCF, the Rust governance model are all variants.



TIP

For a new small project, start as a BDFL. As contributors join, document how decisions are actually made even if it's just “I (the founder) decide; if you disagree, file an issue and we'll discuss.” Implicit governance is the worst kind.

Learning Resources

Topic	Resource
License selector	choosealicense.com
OSI-approved list	opensource.org/licenses
Code of Conduct	Contributor Covenant 2.1
Changelog convention	Keep a Changelog
Semantic versioning	semver.org
Open source community guides	opensource.guide
Crediting all contributors	All Contributors spec
Maintainer perspective (book)	Working in Public — Nadia Eghbal
Funding maintenance	GitHub Sponsors , Open Collective , Tidelift

Suggested Repositories

The “repo” this week is *your own*.

For inspiration on excellent README/CONTRIBUTING quality

- [expressjs/express](https://expressjs.com) — Mature, well-organized contributor docs.

Your Tasks for the Week

- 1. Pick the license.** For your own personal repo, write a justification in your journal on: which license, and why. Add the LICENSE file.
- 2. Hit 100% community profile.** For one of your repos, complete every item GitHub's "Community Standards" checklist asks for. Screenshot the green page in your journal.
- 3. Write the CONTRIBUTING.md.** Cover: setup, how to run tests, how to file a bug, your PR conventions, your review expectations.
- 4. Document governance.** Even if you're a solo BDFL, write a 1-page [GOVERNANCE.md](#) explaining how decisions are made.
- 5. Audit a real project, then contribute to it.** Pick an OSS project you actually use or are curious about. Using this week's checklist:
 - Check its Community Profile
 - Read its LICENSE
 - Read its CONTRIBUTING.md (if it has one)

Then do one small, real thing: either **open a PR** (a typo fix, a missing example in the README, a doc clarification or anything genuinely useful).

Soft Skill of the Week: Setting Boundaries (Saying No)

Once you have an OSS project that gets even modest traction, you will receive: feature requests you don't want to build, bug reports you don't have time for, PRs that don't fit your vision, demands from people who are not paying you to do anything. Maintainer burnout is the OSS equivalent of an occupational hazard, and the cause is almost always **inability to say no**.

// 6.1 Kind, firm refusal. Three ways

1. The clear no with rationale



NOTE

"Thanks for the proposal! I've thought about adding this and decided not to as it adds API complexity for everyone to serve a smaller use case. If it's important to you, you might fork the project; I'd be happy to link to your fork from the README."

Notes the request, explains the why, offers an alternative.

2. The "not now" no



NOTE

"This is a great idea but it's not on the roadmap for v1. I'd accept a PR if someone wanted to maintain it as a plugin."

Honest about your bandwidth, opens the door for someone else to do the work.

3. The boundary no



NOTE

"I'm not able to provide one-on-one debugging support. If you can post a minimum reproducible example as an issue, I'll take a look when I have time."

Doesn't promise free labor.



IMPORTANT

There's a deeper truth: **an OSS project with bad boundaries dies, because the maintainer burns out**. The kindest thing you can do for your future users is to be a maintainer who's still around in 3 years and that requires saying no.

// 6.2 Resources

- [Nadia Eghbal – Working in Public: The Making and Maintenance of Open Source Software](#)
(the chapter on maintainer-to-user dynamics)